

Link to the drive containing the before and after videos :

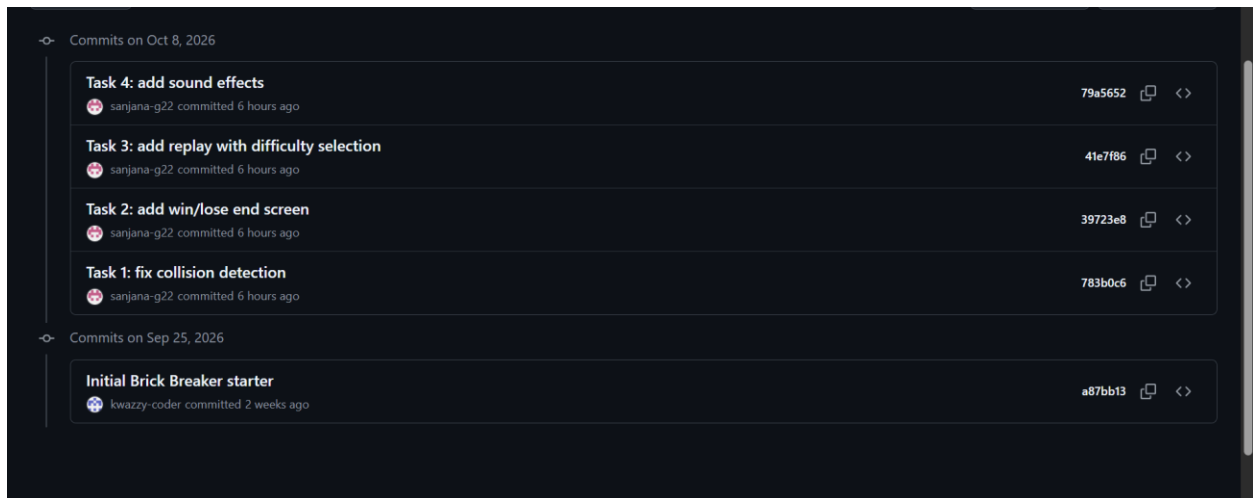
https://drive.google.com/drive/folders/1lJeGpJX1gyKr6osM_WXCJDbyutD8yA3I?usp=sharing

Link to the chat :

<https://claude.ai/share/04bc018e-1ab5-4b79-b8a0-29b244a9b954>

Link to the forked repo which has all the commits task wise :

https://github.com/sanjana-g22/49_brick-breaker



all commits were done task wise in the forked repo but while pushing it to the main lab repos I pushed it at once so the individual commits aren't shown .

Prompts screenshots

1)

I have a Pygame Brick Breaker game and right now the ball always bounces vertically when it hits the paddle or bricks no matter which side it actually hits. When it hits the left or right side of a brick it keeps moving in the same horizontal direction which sometimes makes it clip or skip past nearby bricks. It also sometimes gets stuck jittering on the paddle. Here is the collision code inside `GameEngine.update()`. The Ball Brick and Paddle classes all have x y width and height and the ball also has radius vx and vy. They each have a `rect()` method. Can you fix the collision logic so that brick hits flip vx for left or right side hits and vy for top or bottom hits based on the amount of overlap, the ball gets pushed out of the brick or paddle after a collision so it does not get stuck, the paddle bounce angle depends on where the ball hits the paddle and the ball cannot skip through bricks. Please only change the collision logic and give me just the updated `update()` method .

```
if self.ball.rect().colliderect(self.paddle.rect()):
    self.ball.vy *= -1

for brick in self.bricks:
    if brick.alive and self.ball.rect().colliderect(brick.rect()):
        brick.alive = False
        self.score += 1
        self.ball.vy *= -1
        break
```



2)

Next, the game currently just prints You win or Game over in and then freezes without showing anything on the game window. I want to add a simple end screen inside the game that shows "YOU WIN!" when all the bricks are cleared or "GAME OVER" when there are no lives left, along with the final score and a "Press any key" message. It should then wait for the player to press a key, and the key press should be detected using `handle_event()`. Please keep the changes small and only show me the methods that need to be changed. this is the needed code

```
def handle_event(self, event):
    pass

# inside render():
if self.game_over and not getattr(self, "_game_over_logged", False):
    if self.result == "win":
        print("You win! Final score:", self.score)
    else:
        print("Game over! Final score:", self.score)
    self._game_over_logged = True
```

Show less



3)

next i want to add a replay option after the end screen. the player should be able to choose easy medium or hard using keys 1 2 and 3 where the ball speed and paddle width are different for each difficulty or press esc to quit. replaying should fully reset everything including the bricks score lives ball and paddle. the engine is currently created only once in main.py and _reset_ball() only resets the ball. please add a reset/start method that takes the difficulty setting and update the end screen to show these options. show only the changed code.

game/game_engine.py

```
def __init__(self, width, height):
    self.width = width
    self.height = height

    self.paddle = Paddle(width // 2 - 50, height - 30, 100, 14)

    self.ball = Ball(width // 2, height - 50, radius=8)
    self.ball.vx, self.ball.vy = 4, -4

    self.rows, self.cols = 5, 8
    self.bricks = self._build_bricks(self.rows, self.cols)

    self.lives = 3
    self.score = 0
    self.font = pygame.font.SysFont("Arial", 28)
    self.game_over = False
    self.result = None # "win" or "lose"

def handle_event(self, event):
    if self.game_over and event.type == pygame.KEYDOWN:
        shown_at = getattr(self, "_game_over_time", None)
        if shown_at is not None and pygame.time.get_ticks() - shown_at > 500:
            pygame.event.post(pygame.event.Event(pygame.QUIT))

def _reset_ball(self):
```

```

self.ball.x, self.ball.y = self.width // 2, self.height - 50
self.ball.vx, self.ball.vy = 4, -4

# inside render():
if self.game_over:
    if not getattr(self, "_game_over_logged", False):
        self._game_over_time = pygame.time.get_ticks()
        self._game_over_logged = True

    w, h = screen.get_size()
    overlay = pygame.Surface((w, h), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 180))
    screen.blit(overlay, (0, 0))

    if not hasattr(self, "_end_fonts"):
        self._end_fonts = (pygame.font.Font(None, 72), pygame.font.Font(None, 36))
    big, small = self._end_fonts

    title = "YOU WIN!" if self.result == "win" else "GAME OVER"
    lines = [
        (big, title, (255, 255, 255), h // 2 - 60),
        (small, f"Final score: {self.score}", (255, 255, 255), h // 2),
        (small, "Press any key", (200, 200, 200), h // 2 + 50),
    ]
    for font, text, color, y in lines:
        surf = font.render(text, True, color)
        screen.blit(surf, surf.get_rect(center=(w // 2, y)))
# main.py (game loop)
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.handle_input()
        engine.update()

```

4)

finally i want to add sound effects for brick break paddle hit wall hit and a win or lose sound. i dont have any audio files and i dont want to add any new dependencies so please generate the sounds in code using pygame.mixer and the standard library. if the mixer fails to initialize the game should still run normally without sound. please add a small sound helper and show me where i need to call it in update() and where the win or game over is triggered. show only the changed code.

game/game_engine.py

```
def start(self, difficulty="medium"):
    """Fully (re)initialize the game for the given difficulty."""
    self.difficulty = difficulty
    settings = DIFFICULTIES[difficulty]
    self.speed = settings["speed"]
    # ... paddle, ball, bricks, lives, score, game_over, result reset ...
    self._game_over_logged = False
    self._game_over_time = None

def update(self):
    if self.game_over:
        return

    ball = self.ball
    max_step = max(1.0, ball.radius * 0.5)
    steps = max(1, math.ceil(max(abs(ball.vx), abs(ball.vy)) / max_step))

    for _ in range(steps):
        ball.x += ball.vx / steps
        ball.y += ball.vy / steps

        # ----- Paddle -----
        ball_rect = ball.rect()
        paddle_rect = self.paddle.rect()
        if (ball.vy > 0
            and ball_rect.colliderect(paddle_rect)
            and ball_rect.centery < paddle_rect.centery):
            ball.y -= ball_rect.bottom - paddle_rect.top
            offset = (ball_rect.centerx - paddle_rect.centerx) / (paddle_rect.width / 2)
```

```

        speed = math.hypot(ball.vx, ball.vy)
        ball.vx = speed * math.sin(angle)
        ball.vy = -speed * math.cos(angle)

# ----- Bricks -----
ball_rect = ball.rect()
hit_brick = None
best_area = 0
for brick in self.bricks:
    if brick.alive and ball_rect.colliderect(brick.rect()):
        overlap = ball_rect.clip(brick.rect())
        area = overlap.width * overlap.height
        if area > best_area:
            best_area = area
            hit_brick = brick

if hit_brick is not None:
    brick_rect = hit_brick.rect()
    hit_brick.alive = False
    self.score += 1
    # ... side vs top/bottom push-out and velocity flip ...

# ----- Walls -----
if ball.x - ball.radius <= 0:
    ball.x = ball.radius
    ball.vx = abs(ball.vx)
elif ball.x + ball.radius >= self.width:
    ball.x = self.width - ball.radius
    ball.vx = -abs(ball.vx)
if ball.y - ball.radius <= 0:
    ball.y = ball.radius
    ball.vy = abs(ball.vy)

# ----- Lives / win -----
if ball.y - ball.radius > self.height:
    self.lives -= 1
    if self.lives <= 0:
        self.game_over = True
        self.result = "lose"

```

Prompts

1) I have a Pygame Brick Breaker game and right now the ball always bounces vertically when it hits the paddle or bricks no matter which side it actually hits. When it hits the left or right side of a brick it keeps moving in the same horizontal direction which sometimes makes it clip or skip past nearby bricks. It also sometimes gets stuck jittering on the paddle. Here is the collision code inside `GameEngine.update()`. The `Ball` `Brick` and `Paddle` classes all have `x y width` and `height` and the ball also has `radius vx` and `vy`. They each have a `rect()` method. Can you fix the collision logic so that brick hits flip `vx` for left or right side hits and `vy` for top or bottom hits based on the amount of overlap, the ball gets pushed out of the brick or paddle after a collision so it does not get stuck, the paddle bounce angle depends on where the ball hits the paddle and the ball cannot skip through bricks. Please only change the collision logic and give me just the updated `update()` method .

```

if self.ball.rect().colliderect(self.paddle.rect()): self.ball.vy *= -1
for brick in self.bricks:
    if brick.alive and self.ball.rect().colliderect(brick.rect()):
        brick.alive = False

```

```
self.score += 1
self.ball.vy *= -1
break
```

2) Next, the game currently just prints You win or Game over in and then freezes without showing anything on the game window. I want to add a simple end screen inside the game that shows "YOU WIN!" when all the bricks are cleared or "GAME OVER" when there are no lives left, along with the final score and a "Press any key" message. It should then wait for the player to press a key, and the key press should be detected using `handle_event()`. Please keep the changes small and only show me the methods that need to be changed. this is the needed code

```
def handle_event(self, event): pass
# inside render():
if self.game_over and not getattr(self, "_game_over_logged", False):
if self.result == "win":
print("You win! Final score:", self.score)
else:
print("Game over! Final score:", self.score)
self._game_over_logged = True
```

3) next i want to add a replay option after the end screen. the player should be able to choose easy medium or hard using keys 1 2 and 3 where the ball speed and paddle width are different for each difficulty or press esc to quit. replaying should fully reset everything including the bricks score lives ball and paddle. the engine is currently created only once in main.py and

_reset_ball() only resets the ball. please add a reset/start method that takes the difficulty setting and update the end screen to show these options. show only the changed code.

game/game_engine.py

```
def __init__(self, width, height):
    self.width = width
    self.height = height
    self.paddle = Paddle(width // 2 - 50, height - 30, 100, 14)
    self.ball = Ball(width // 2, height - 50, radius=8)
    self.ball.vx, self.ball.vy = 4, -4
    self.rows, self.cols = 5, 8
    self.bricks = self._build_bricks(self.rows, self.cols)
    self.lives = 3
    self.score = 0
    self.font = pygame.font.SysFont("Arial", 28)
    self.game_over = False
    self.result = None # "win" or "lose"
    def handle_event(self, event):
        if self.game_over and event.type == pygame.KEYDOWN:
            shown_at = getattr(self, "_game_over_time", None)
            if shown_at is not None and pygame.time.get_ticks() - shown_at > 500:
                pygame.event.post(pygame.event.Event(pygame.QUIT))
        def _reset_ball(self):
            self.ball.x, self.ball.y = self.width // 2, self.height - 50
            self.ball.vx, self.ball.vy = 4, -4
        # inside render():
        if self.game_over:
            if not getattr(self, "_game_over_logged", False):
                self._game_over_time = pygame.time.get_ticks()
                self._game_over_logged = True
            w, h = screen.get_size()
            overlay = pygame.Surface((w, h), pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 180))
            screen.blit(overlay, (0, 0))
            if not hasattr(self, "_end_fonts"):
                self._end_fonts = (pygame.font.Font(None, 72), pygame.font.Font(None, 36))
            big, small = self._end_fonts
            title = "YOU WIN!" if self.result == "win" else "GAME OVER"
            lines = [
                (big, title, (255, 255, 255), h // 2 - 60),
                (small, f"Final score: {self.score}", (255, 255, 255), h // 2),
                (small, "Press any key", (200, 200, 200), h // 2 + 50),
```

```

]
for font, text, color, y in lines:
    surf = font.render(text, True, color)
    screen.blit(surf, surf.get_rect(center=(w // 2, y)))
main.py (game loop)
engine = GameEngine(WIDTH, HEIGHT)
def main(): running = True while running: for event in pygame.event.get(): if event.type ==
pygame.QUIT: running = False engine.handle_event(event)
engine.handle_input()
engine.update()
engine.render(SCREEN)
pygame.display.flip()
clock.tick(FPS)
pygame.quit()

```

4) finally i want to add sound effects for brick break paddle hit wall hit and a win or lose sound. i dont have any audio files and i dont want to add any new dependencies so please generate the sounds in code using pygame.mixer and the standard library. if the mixer fails to initialize the game should still run normally without sound. please add a small sound helper and show me where i need to call it in update() and where the win or game over is triggered. show only the changed code.

```

game/game_engine.py
def start(self, difficulty="medium"):
    """Fully (re)initialize the game for the given difficulty."""
    self.difficulty = difficulty
    settings = DIFFICULTIES[difficulty]
    self.speed = settings["speed"]
    # ... paddle, ball, bricks, lives, score, game_over, result reset ...
    self._game_over_logged = False
    self._game_over_time = None
    def update(self):
    if self.game_over:
    return

```

```

ball = self.ball
max_step = max(1.0, ball.radius * 0.5)
steps = max(1, math.ceil(max(abs(ball.vx), abs(ball.vy)) / max_step))
for _ in range(steps):
    ball.x += ball.vx / steps
    ball.y += ball.vy / steps
    # ----- Paddle -----
    ball_rect = ball.rect()
    paddle_rect = self.paddle.rect()
    if (ball.vy > 0
        and ball_rect.colliderect(paddle_rect)
        and ball_rect.centery < paddle_rect.centery):
        ball.y -= ball_rect.bottom - paddle_rect.top
        offset = (ball_rect.centerx - paddle_rect.centerx) / (paddle_rect.width / 2)
        offset = max(-1.0, min(1.0, offset))
        angle = offset * math.radians(60)
        speed = math.hypot(ball.vx, ball.vy)
        ball.vx = speed * math.sin(angle)
        ball.vy = -speed * math.cos(angle)
    # ----- Bricks -----
    ball_rect = ball.rect()
    hit_brick = None
    best_area = 0
    for brick in self.bricks:
        if brick.alive and ball_rect.colliderect(brick.rect()):
            overlap = ball_rect.clip(brick.rect())
            area = overlap.width * overlap.height
            if area > best_area:
                best_area = area
                hit_brick = brick
    if hit_brick is not None:
        brick_rect = hit_brick.rect()
        hit_brick.alive = False
        self.score += 1
    # ... side vs top/bottom push-out and velocity flip ...
    # ----- Walls -----
    if ball.x - ball.radius <= 0:
        ball.x = ball.radius
        ball.vx = abs(ball.vx)
    elif ball.x + ball.radius >= self.width:
        ball.x = self.width - ball.radius
        ball.vx = -abs(ball.vx)
    if ball.y - ball.radius <= 0:
        ball.y = ball.radius

```

```
ball.vy = abs(ball.vy)
# ----- Lives / win -----
if ball.y - ball.radius > self.height:
    self.lives -= 1
if self.lives <= 0:
    self.game_over = True
```

```
self.result = "lose"
```

```
else:
```

```
self._reset_ball()
```

```
if all(not b.alive for b in self.bricks):
```

```
self.game_over = True
```

```
self.result = "win"
```

The paddle hit, brick hit, wall hit and win/lose events are all inside update() above, and start() is where the game resets on replay.

All the changes were made to the game_engine file

Bugs found

1. Wrong collision bounce. The ball always flipped its vertical direction on any hit, so side hits on bricks and the paddle bounced wrongly.
2. Ball clipping and sticking. The ball could stay inside a brick or the paddle and jitter, and could skip past neighboring bricks at speed.
3. No end screen. Winning or losing only printed to the console, and the game window just froze.
4. No replay. The game couldn't be restarted, and there was no difficulty choice.
5. No sound. The game had no audio feedback.

Fixes and features

1. Collision detection: the bounce side is now worked out from the overlap, so side hits flip the horizontal direction and top/bottom hits flip the vertical one. The ball is pushed out after each hit, the paddle bounce angle depends on where the ball lands, and the ball moves in small steps so it can't skip bricks.
2. Game over screen: an in-window "YOU WIN!" or "GAME OVER" screen shows the final score and waits for a key press.
3. Replay with difficulty: after the end screen, press 1 (Easy), 2 (Medium) or 3 (Hard) to play again, with different ball speed and paddle width, or press Esc to quit. Everything resets on replay.
4. Sound effects: brick break, paddle hit, wall hit and win/lose sounds, all generated in code with no audio files, and the game still runs silently if audio isn't available.